Department of Computer Science and Engineering, BUET

Prompt Tuning

Task Specific Prompt Learning

CSE 200

2305068 Sk. Arib Rajin Shahan **2305075** Sadman Zaman **2305076** Fatin Nehal Zubaeer

PART 1

Introduction to Prompt Tuning

Freeze the billions. Learn a handful of vectors and put them in front.

What is Prompt Tuning?

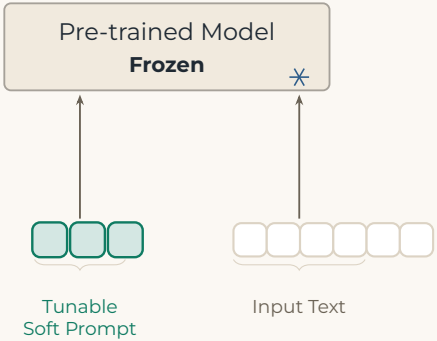
Prompt tuning is the process of adapting LLMs to new tasks by training a small set of parameters known as **prompts**.

These prompts are **prepended to the input text** to guide the LLM towards generating the desired output.

Learn the prompt instead of writing it

Prompt tuning is a **parameter-efficient fine-tuning** technique. It adapts a large pretrained model to a new task *without* updating its billions of parameters.

Instead it learns a small set of trainable vectors, **soft prompts**, or virtual tokens inserted into the model's input space.

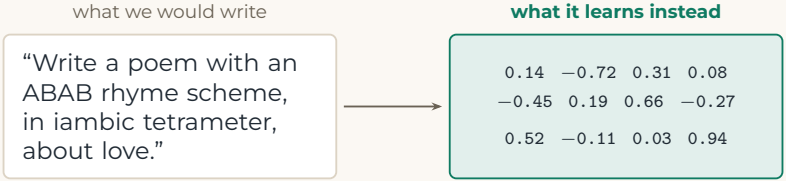


Hard Prompts and Soft Prompts

	Hard prompt	Soft prompt
Representation	Human-written tokens	Learnable vectors
Space	Vocabulary space	Embedding space
Optimization	Trail and error	Gradient descent
Readable	Yes	No
Portable	Across models	Tied to model's embeddings

Soft prompts guide the model more reliably than hand-written text, and they work well when training data is limited.

An analogy: the instruction we never have to write



Show the model enough example poems and it learns the pattern, rhyme scheme, metre, subject as numbers. The analogy holds for what the prompt *does*; the real vector has no fields you could read off like this.

Benefits of Prompt Tuning

1. Efficient

Prompt tuning requires training only a small number of prompt parameters, resulting in faster adaptation to new tasks.

2. Flexible

Prompt tuning can be applied to a wide range of tasks in various domains, including natural language processing, image classification, and code generation.

3. Interpretable

With prompt tuning, it is possible to inspect the prompt parameters to understand how the LLM is being guided towards generating the desired output.

Cycle of Prompt Tuning Benefits

Scale Solutions

Implement AI across industries efficiently.

Optimize Prompts

Refine input prompts for better results.

Reduce Costs

Lower resource usage and expenses.

Enhance Accuracy

Improve model accuracy and context awareness.

Fine-Tuning Required a New Model for Every Task

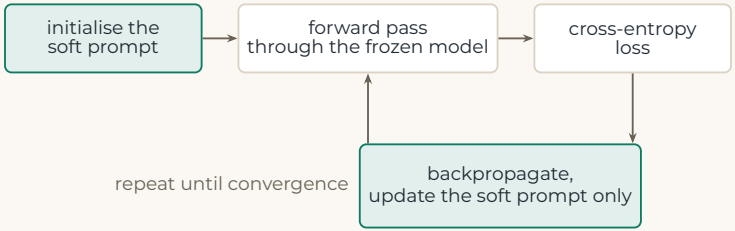
Fine-tuning adapts a pretrained model by updating all of its weights on a single task's data. Once tuned, those weights are specialized to that task, they can't also serve another task well. So each new task requires its own full copy of the model. For a model like T5-XXL, that means duplicating all 11 billion parameters for every single task.

PART 2

How a soft prompt is trained

Optimizing Learnable Prompt Representations While Keeping the Base Model Frozen.

Soft Prompt Training Workflow

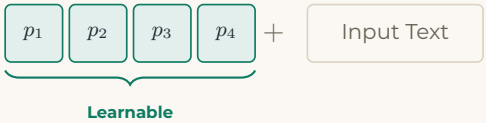


Soft prompt tuning follows the standard neural-network training loop. The fundamental difference is that only the soft prompt is updated, while the pretrained model remains unchanged.

Initialising the Soft Prompt

A soft prompt is a small set of learnable embedding vectors that are prepended to the input text.

- Initialise the prompt with a small number of vectors
- The vectors can be initialised using:
  - random values
  - existing token embeddings
  - sampled vocabulary embeddings
- The pretrained model parameters remain **frozen**



Forward Pass and Cross-Entropy Loss

The soft prompt is combined with the input text and passed through the pretrained model.

- The model processes the **soft prompt + input text**
- The model parameters are not modified
- The model produces a prediction
- **Cross-entropy loss** measures the difference between the prediction and the target output

Backpropagation and Prompt Optimization

The loss is backpropagated through the entire network to determine how the soft prompt contributed to the error.

- **Gradients** are computed through the frozen model
- The model weights remain unchanged
- The optimizer uses the gradient with respect to the soft prompt
- Only the **soft-prompt parameters** are updated

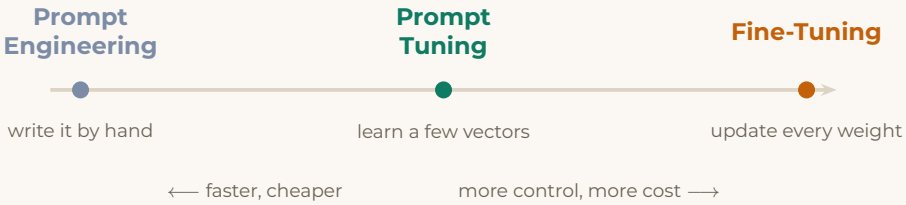
PART 3

When Prompt Tuning Fits

Efficiently adapting pretrained models to new tasks with minimal parameter updates.

Choosing Between Prompt Tuning and Fine Tuning

Prompt tuning sits in between prompt engineering and full fine tuning: it buys **behavioural consistency** without the full cost of updating every weight.



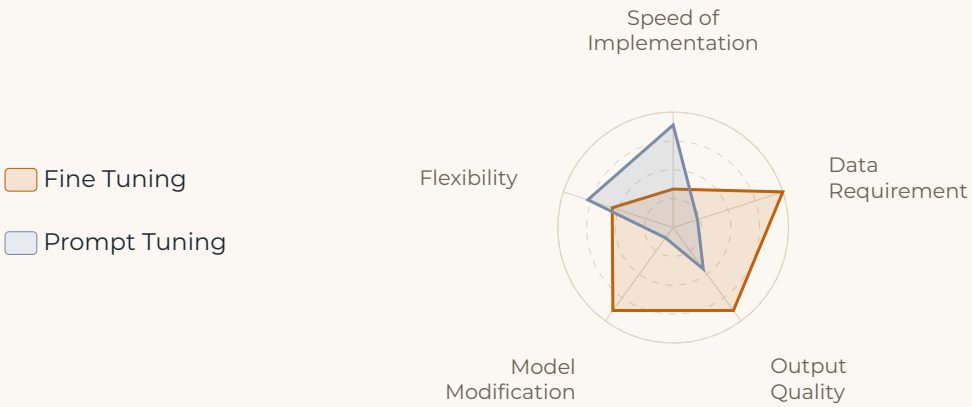
When to Reach for Prompt Tuning

- **Parameter efficiency:** Adapt the model by training only a small set of prompt parameters.
- **Task adaptation:** Useful for adapting a pretrained model to a specific downstream task.
- **Multiple tasks:** Effective when maintaining separate lightweight task-specific configurations.
- **Preserve the base model:** Suitable when we want to keep the original model unchanged.

When to Reach for Full Fine Tuning

- **Domain specialisation:** Adapt the model to a specific domain, industry, or knowledge area
- **Complex tasks:** Best for tasks requiring deeper changes to the model's behaviour or capabilities
- **Large labelled datasets:** More suitable when substantial task-specific training data is available
- **Higher performance:** Use when maximising task-specific accuracy matters more than training cost

Fine Tuning and Prompt Tuning: A Comparative Study



PART 4

Worked Example

Prompt tuning a frozen model, one gradient step at a time.

The Scenario

A pretrained model already understands English fluently. We want it to classify movie reviews as **Positive** or **Negative** without touching a single one of its weights.

Step 1: Defining the Training Task

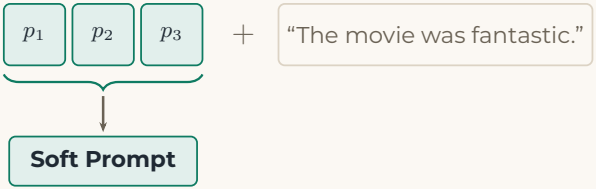
A handful of labelled examples is enough to start.

Input	Target
The movie was fantastic.	**Positive**
The acting was terrible.	**Negative**
I really enjoyed this film.	**Positive**
The story was boring.	**Negative**

The model already knows language – we only need to teach it how to use that knowledge for sentiment.

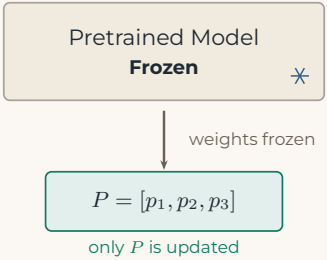
Step 2: Create the Soft Prompt

We use **3 soft-prompt tokens**: $P = [p_1, p_2, p_3]$, initialised as random vectors in the model's embedding space, not words.



Step 3: Freeze the Pretrained Model

The pretrained model's weights, θ, remain **completely frozen** throughout training, preserving everything it learned during pretraining. Only the soft-prompt parameters $P = [p_1, p_2, p_3]$ are trainable and updated during optimisation.

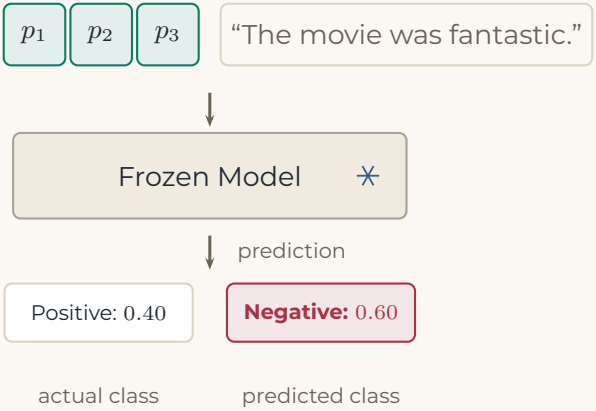


Step 4: Forward Pass

The soft prompt is prepended to the input and passed through the **frozen model** to produce a prediction. The prediction is only used to compute the loss next.

- Input: $[p_1, p_2, p_3]$ + “The movie was fantastic.”
- Predicted: Positive 0.40, Negative 0.60
- But the correct answer is **Positive** – the model is current making a wrong decision

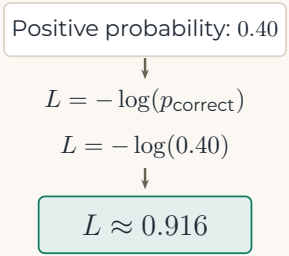
An Incorrect Initial Prediction



Step 5: Calculate the Loss

We use cross-entropy loss. The correct class is Positive, whose predicted probability was 0.40.

- $L = -\log(p_{\text{correct}})$
- $L = -\log(0.40) = -(-0.9163)$
- $L \approx \mathbf{0.916}$



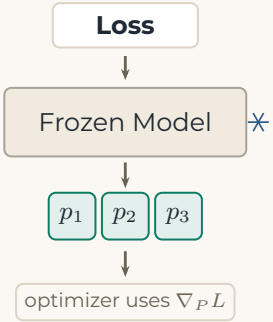
Step 6: Backpropagation

The loss is backpropagated through the entire network to determine how each soft-prompt vector contributed to the error.

$$\nabla_P L = \frac{\partial L}{\partial P}$$

- Gradients are computed through the frozen model
- $\theta_{\text{model}} \rightarrow$ frozen, $\quad P \rightarrow$ trainable
- Only $\nabla_P L$, the gradient with respect to P, is kept

For our example: $\nabla_P L \approx [\,0.20,\ -0.10,\ 0.30\,]$



Step 7: Update the Soft Prompt

With learning rate $\eta = 0.1$, the optimizer moves P in the direction that **reduces the loss**:

$$P_{\text{new}} = P_{\text{old}} - \eta \nabla_P L \qquad \Longrightarrow \qquad P_{\text{new}} = P_{\text{old}} + \begin{bmatrix} -0.02 \\ +0.01 \\ -0.03 \end{bmatrix}$$

- p_1 moves by -0.02
- p_2 moves by $+0.01$
- p_3 moves by -0.03

The minus sign moves P toward lower loss, not higher. Only the soft prompt moves in embedding space; the pretrained model remains completely frozen.

Steps 8 & 9: Repeat and Learn

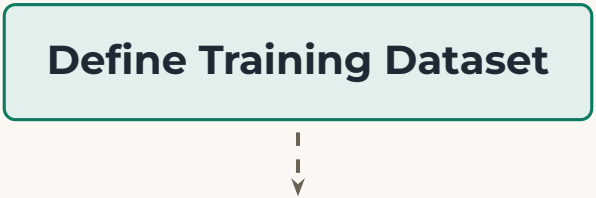
Step 8: Repeat the Steps

After updating the soft prompt, we move to the next example. The same cycle repeats: forward pass through the frozen model, loss against the target, backpropagation to find how P should change.

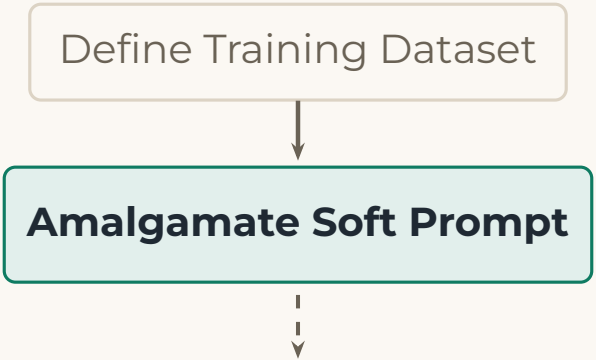
Step 9: The Prompt Learns

Initially, P holds random vectors, so the model produces incorrect predictions and **high loss**. As gradient updates accumulate, these vectors are gradually shaped into a useful, task-specific representation.

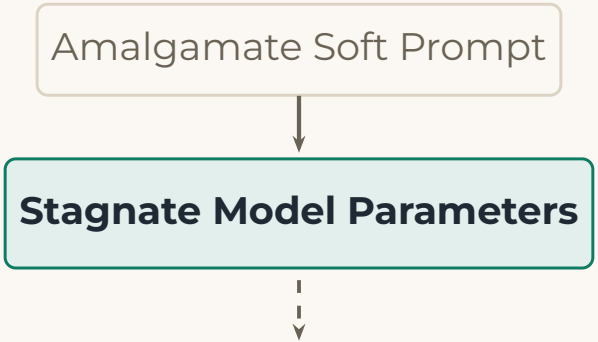
Workflow 1: Define the Training Task



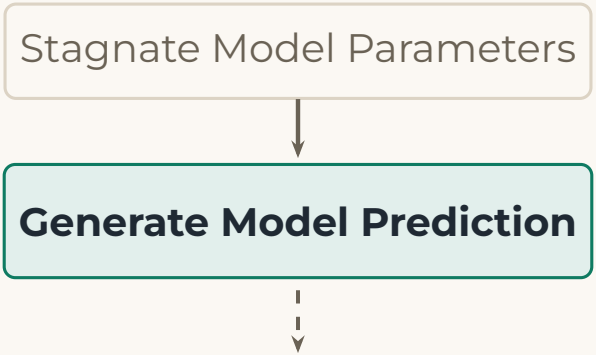
Workflow 2: Initialise the Soft Prompt



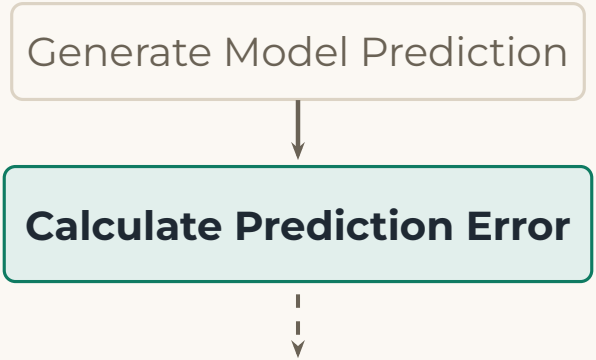
Workflow 3: Freeze the Pretrained Model



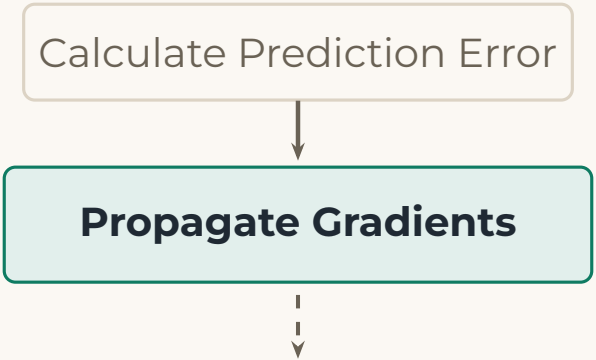
Workflow 4: Forward Pass



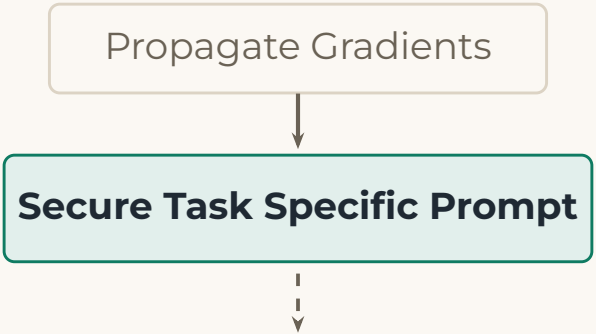
Workflow 5: Compute the Loss



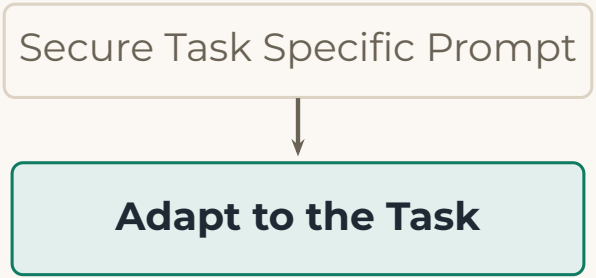
Workflow 6: Backpropagation



Workflow 7: Update the Soft Prompt



Workflow 8: The Prompt Learns



PART 5

Overcoming Challenges

Practical techniques for writing prompts the model can actually follow.

Approaching Challenges

- **Identify the core request:** Begin by pinning down the primary question or action you need from the model.
- **Break it down:** If the prompt contains multiple parts, split it into smaller, focused components, one task per piece.
- **Prioritize information:** Decide which details are essential and which are secondary; keep the prompt focused on the essentials.
- **Use clear language:** Avoid jargon, ambiguous terms, or overly technical phrasing – write so anyone could follow it.
- **Remove unnecessary information:** Strip out context, backstory, or details that don't directly serve the core request.

Approaching Challenges

- **Restructure the prompt:** Reorganize simplified components into a clear, concise prompt – bullet points or numbered lists help separate distinct tasks.
- **Avoid double negatives:** Complex negations confuse the model, state what is required using positive language instead.
- **Specify constraints:** If there are topics or references to avoid, state them explicitly in the prompt.
- **Provide context:** Include context where it is genuinely needed so the model understands the background of the request.
- **Test for clarity:** Have someone else review the prompt – those questions reveal what still needs simplifying. Split long, convoluted asks into separate prompts rather than one dense one.

References

[1] B. Lester, R. Al-Rfou, N. Constant. *The Power of Scale for Parameter-Efficient Prompt Tuning*. EMNLP 2021.

[2] X. L. Li, P. Liang. *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. ACL 2021.

[3] T. Brown et al. *Language Models are Few-Shot Learners*. NeurIPS 2020.

[4] C. Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR 2020.

[5] N. Houlsby et al. *Parameter-Efficient Transfer Learning for NLP*. ICML 2019.

[6] K. Hambardzumyan, H. Khachatrian, J. May. *WARP: Word-level Adversarial ReProgramming*. ACL 2021.

CONCLUSION

Thank You